What to eat! Requirements

Version 1.1 | October 27, 2026 | Juan Castro | Meta Global AI Developer Hackathon, Indie track

Overview

What to eat! is the middleman between you and your food decision. It is a chat: short questions, three real places on a map, and an app that gets better at reading you with every meal. Algorithms predict; Muse Spark (Meta Model API) handles language. Google Places supplies the restaurants; Supabase stores history and the learned profile.

Every recommendation traces to data. When evidence is thin, the app asks instead of assuming. Goals: efficient, cheap, accurate.

Core rules

ID	Requirement	Phase
R1	Minimal UI with a glassy visual style (direction TBD). Every piece of text is as short as possible: one line per message. Recommendation cards show name, distance, price, and a one-line reason.	All
R2	Only places returned by Google Places are scored and shown. Muse never adds, removes, or reorders places.	1
R3	Log from day one: time, day of week, meal slot, answers, companions, price level, distance, the top 3, and the final choice. Phase 2 models depend on this data.	1

Functional requirements

Onboarding (taste quiz)

ID	Requirement	Phase
FR1	About 4 taps: favorite categories and hard no's (both from the ML2 list), typical budget per own meal. Answers form the quiz prior. Hard no's are a hard filter, never shown.	1
FR2	Quiz asks whether the user mostly eats solo or with people. If with people, the user adds companions by name, which fills the companions table.	1

Chat and questions

ID	Requirement	Phase
FR3	The app is a chat. One question per message, drawn from a pool: craving (category), budget, distance, who you're with.	1
FR4	Exactly 3 questions (3 messages) before the first round of recommendations. The ML6 confidence gate chooses which 3; Muse only phrases them. Answers the models can predict are pre-filled and editable. Cold start (Phase 1): the gate runs on the quiz prior.	1-2
FR5	Free-text answers are parsed by Muse into the same structured answers as quick-reply chips (ML7).	1

Recommendations

ID	Requirement	Phase
FR6	Google Places nearby search from the current location, using the search radius defined in FR-R2.	1
FR7	Each Places candidate is scored deterministically (ML5); the top 3 are shown, each with a one-line reason written by Muse from the actual evidence (ML7).	1
FR8	The top 3 appear as map pins and compact cards; tapping one opens directions.	1
FR9	The user confirms which place they chose, or none.	1
FR9b	After the first round, the conversation stays open as long as needed; the user can refine (cheaper, closer, something else) and get new picks.	1

Feedback

ID	Requirement	Phase
FR10	A local notification fires 2 hours after the last app activity, only once a decision has been made. New activity reschedules it.	1
FR11	"Went?" detection on app open: a foreground location check within 250 m of the picked place, between pick time and pick + 4 h, counts as a likely visit. If unsure, a pop-up asks if the user went to their last pick: Yes / No. Yes leads to "How was it?" with 5 stars. No leads to "Thanks!". No other feedback questions.	1
FR12	A preference setting for chat input: text (default) or voice. Voice uses Muse Voice Transcribe, then Muse parses the transcript into structured answers (ML7).	3
FR13	Went and stars update the inputs of the cuisine and place models (ML3, ML4). No free-text profile rewrite.	2

Background location is out of scope: it needs a development build and a paid Apple account.

Companions

ID	Requirement	Phase
FR14	Per companion: meal count, likely days of the week, and most likely categories together, derived from the same meal counts as ML3.	2
FR15	Selected companions are ML1 context: they change which questions are asked (ML6) and how places score (ML3).	2

Insights

ID	Requirement	Phase
FR16	Charts: price by meal slot (amount paid for backfill, Places price level for live meals), most common category by meal slot, distance traveled, meals by companion, top categories.	2
FR17	A one-line Muse-written takeaway at the top of the Insights screen, phrased from computed stats only.	2

Prediction model

Algorithms predict; Muse only handles language. Every value marked "starting value" is tuned only on live data, never on the backfill.

ID	Requirement	Phase
ML1	Context features: meal slot (breakfast, lunch, dinner, late), weekday or weekend, companions (live data only for now).	1
ML2	Fixed category list, used by backfill, models, and Google Places type mapping: Burgers, Pizza, Mexican, Latin, Chinese, Japanese/Sushi, Korean, Thai, Vietnamese, Indian, Mediterranean, Italian, American/Diner, BBQ, Chicken/Wings, Sandwiches, Seafood, Salads/Healthy, Breakfast/Brunch, Coffee/Bakery, Dessert. Places that map to none go to Other, which is shown but never predicted.	1
ML3	Layer 1, cuisine model: recency-weighted counts per context. Backs off to broader contexts when data is thin: this companion, then user overall, then quiz prior. Outputs a probability per category plus the number of meals behind it.	2
ML4	Layer 2, place model: return score from visits, recency, and 1-5 star ratings; 1-2 stars drop it sharply. New places are scored on price fit, distance fit, and Google Places rating.	2
ML5	Scoring: each real Places candidate is scored deterministically from ML3, ML4, price fit, and distance fit. Top 3 = 2 familiar picks + 1 discovery slot (best candidate outside the user's usual categories). Phase 1 uses the quiz prior in place of ML3 and ML4.	1

ID	Requirement	Phase
ML6	Confidence gate: if the top prediction is weak or rests on too few meals, the app asks a question instead of assuming. This decides which of the 3 questions are asked.	2
ML7	Muse is language only: parse free text into structured answers, write each pick's one-line reason from the actual evidence, run the conversation. Muse never reorders, removes, or adds places. One Muse call writes all 3 reasons; its output is checked against the candidate IDs.	1
ML8	Funnel logged per recommendation: shown, interested (card or directions tapped), picked, went, stars.	1
ML9	Evaluation: time-ordered replay on backfilled history (predict each meal using only earlier meals). Report cuisine and place top-3 hit rates against two baselines (always most frequent; random), with sample size stated.	2

Starting values

Item	Starting value
Meal slot	Breakfast before 11:00; lunch 11:00 to 16:00; dinner 16:00 to 21:00; late after 21:00.
Likely visit (FR11)	Within 250 m of the picked place, between pick time and pick + 4 h.
Usual categories (ML5)	Categories with at least 15% of the user's recency-weighted meals. Until data exists: quiz favorites.
Weak prediction (ML6)	Top probability under 0.4, or fewer than 5 meals behind it (matches FR-R3).
Price levels	$ = under $15; $$ = $15 to $30; $$$ = $30 to $60; $$$$ = over $60 (per own meal).
Price fit	Compares levels: candidate at or below the budget level = full fit; one level above = half; two or more above = none; unknown = half.

Data

- Price everywhere means what the user paid for their own meal. Backfilled meals keep the real amount, converted to a level with the table above. Live meals store the Places price level; price paid stays null.
- Backfill: Juan's last 30 days of eating out, entered as CSV with columns date, meal_slot, place, category, price, companions. Companions are filled only when sure. Missing values stay null, never guessed.
- On import, each place name is resolved once to a Google place_id so backfilled and live visits match. Ambiguous matches are confirmed by Juan.

- Backfill tests Layers 1 and 2 by meal slot and day. Companion patterns and star-based return scores learn from live use only. Backfill has no location, so it does not feed the search radius.
- ML9 place hit rate: the candidate pool for each replayed meal is the set of places seen in earlier meals. A first visit to a new place counts as a miss; the share of new-place meals is reported separately. Parameters are fixed before replay, not tuned on it.

Adaptive search radius

The search area should grow more accurate to the user as usage increases. Values below are proposed starting points, to be tuned against real usage data.

ID	Requirement	Phase
FR-R1	Record the straight-line distance from the session location to the chosen place for every confirmed meal.	1
FR-R2	Radius for a new search = the recency-weighted 80th percentile of past travel distances in a similar context (ML1: meal slot, weekday or weekend, companions), clamped between 1 and 25 miles.	2
FR-R3	Cold start: use the quiz answer until 5 confirmed meals exist for that context.	1-2
FR-R4	If fewer than 8 candidates come back, expand the radius by 1.5x, up to two times.	2

Non-functional requirements

ID	Requirement	Phase
NFR1	Runs on iPhone through Expo Go; no App Store build required.	1
NFR2	Backend deployed to the cloud by the end of Phase 1, so the app works away from home Wi-Fi.	1
NFR3	API keys live only on the backend; every API request requires a valid Supabase JWT.	1
NFR4	First round of recommendations in under about 8 seconds after the third answer: one Places search and one Muse call per round.	1

Architecture

Mobile: Expo (React Native, TypeScript). **Backend:** FastAPI (Python, uv). **Places:** Google Places API. **Prediction:** deterministic scoring and models in the backend (ML3 to ML6). **AI:** Muse Spark via Meta Model API for language only (ML7); Muse Voice Transcribe for voice input. **Data and auth:** Supabase (Postgres).

Tables: users, taste_profiles, companions, sessions, session_companions, messages, feedback, meals (source = backfill or live), categories, places, recommendation_events.

Phases and deliverables

Phase	Scope	Due
1	**Core loop** (Tue Oct 27 to Thu Oct 29). Deliverable: the app in daily use on iPhone by Thursday night. Quiz, chat with 3 questions, scored top 3 on a map with Muse reasons, refinement, confirm, Went? + stars with notification, full logging and funnel events, deployed backend.	Oct 29
2	**Learning and insights** (Fri Oct 30 to Sun Nov 1). Deliverable: the app visibly adapts to real data. Backfill import, cuisine and place models, confidence gate, offline evaluation, adaptive radius, companion context, Insights charts.	Nov 1
3	**Voice and demo** (Mon Nov 2 to Wed Nov 4). Deliverable: project submitted by 9:00 PM ET on Nov 4. Voice input for chat answers, edge cases, polish, README with evaluation results, demo video.	Nov 4

Risks

- Phase 1 is the tightest phase and includes a cloud deploy. Every day it slips removes a self-use day, which is the core of the demo. First cut if needed: FR9b moves to Phase 2.
- Voice is deliberately Phase 3. If time runs out, text input still delivers the full product.
- 30 days of backfill is roughly 30 to 60 meals. Evaluation results carry wide uncertainty and are reported with sample size, not as proof.
- The radius thresholds (80th percentile, 1 to 25 miles, 8 candidates) and all starting values above are untested guesses, to be tuned after 2 to 3 days of live data.

Changes from v1.0

- New Prediction model section (ML1 to ML9), starting values, and data rules.
- R1: minimal glassy UI. R2: Muse never adds, removes, or reorders places. R3 adds meal slot.
- FR1: categories from ML2, spice level dropped, hard no's are a hard filter. FR3: dine-in vs takeout dropped.
- FR4: questions chosen by the ML6 gate. FR5: replaced by free-text parsing (rule-based fallback folded into FR4 cold start).
- FR7: deterministic scoring; Muse writes reasons only.
- FR10, FR11: replaced by the Went? + stars flow. FR12: voice becomes input for chat answers. FR13: feedback updates models.
- FR14, FR15, FR16, FR-R2: aligned with ML1 context and the price rules.